

Informática Forense

6. Análisis forense con Volatility

PRESENTADO POR:
David Santo Orcero

ENTIDAD
Perito judicial

MÓDULO
Avanzado

VOLATILITY



Introducción

Volatility es una herramienta de análisis forense de memoria RAM, escrita en Python, que permite extraer información relevante de volcados de memoria.

Dos versiones (Volatility 2 y Volatility 3) que operan en la práctica como programas distintos.



¿Qué podemos hacer con Volatility?

- Listar procesos activos
- Detectar rootkits
- Extraer contraseñas y sesiones
- Obtener conexiones de red
- **Herramienta clave para forense en caliente y DFIR**



Lenguaje y Arquitectura

Aspecto	Volatility 2	Volatility 3
Lenguaje	Python 2.7	Python 3.x
Arquitectura	Más rígida	Modular y extensible
Definir perfil	Requiere	No necesario



Soporte y Compatibilidad

Sistema / Soporte	Volatility 2	Volatility 3
Sistemas antiguos (Win XP, 7)	Completo	Limitado
Sistemas nuevos (Win10, Linux)	Parcial	Mejorado
Linux y MacOS modernos	Limitado	Mejor soporte



Plugins y Desarrollo

○ **Volatility 2:**

- Muchos plugins existentes y maduros.
- Comunidad amplia.

○ **Volatility 3:**

- Arquitectura moderna y orientada a desarrolladores.
- Mejor documentación para crear plugins.
- Aún en desarrollo activo.



Uso y Experiencia

Aspecto	Volatility 2	Volatility 3
Requiere perfil manual	Sí	No
Auto detección de SO	No	Sí
Modularidad	Parcial	Total



Comparativa Final

Característica	Volatility 2	Volatility 3
Estabilidad	Muy estable	En desarrollo
Soporte sistemas nuevos	Limitado	Mejorado
Requiere perfil	Sí	No
Extensibilidad	Menor	Modular
Lenguaje	Python 2.7	Python 3.x



Es decir...

- Usaremos **Volatility 2** si trabajamos con:
 - Sistemas antiguos (Windows XP, 7).
 - Plugins clásicos ampliamente documentados.
- Usaremos **Volatility 3** si:
 - Trabajamos con Windows 10/11 o Linux moderno.
 - Nos valen ambos.

VOLATILITY 2



Instalación de Python 2.7

```
apt update
sudo apt install -y build-essential zlib1g-dev
libncurses5-dev libgdbm-dev \
    libnss3-dev libssl-dev libreadline-dev
    libffi-dev wget
wget https://www.python.org/ftp/python/2.7.18/
    Python-2.7.18.tgz
tar -xvzf Python-2.7.18.tgz
cd Python-2.7.18
./configure --prefix=/opt/python2
make -j$(nproc)
sudo make install
```



Instalación de Volatility 2 (Python 2.7)

```
export PATH="/opt/python2/bin:$PATH"
apt update
apt install git pcregrep
curl https://bootstrap.pypa.io/pip/2.7/get-pip.py
    --output get-pip.py
python2 get-pip.py
pip2 install distorm3
pip2 install pycryptodome
git clone https://github.com/volatilityfoundation
    /volatility.git
cd volatility
python2.7 setup.py install
```



Comando básico con Volatility

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 pslist
```

Ojo: no todos los comandos funcionan con todos los perfiles.



Identificar el perfil

```
python2.7 vol.py -f memoria.raw imageinfo
```



Listar procesos

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 pslist
```



Buscar conexiones de red

```
python2.7 vol.py -f memoria.raw --profile=Win7SP1  
x64 netscan
```



Extraer comandos ejecutados

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 cmdscan
```



Detección de malware

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 malfind
```



Extraer archivos

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 dumpfiles -D ./ficherosvolcados
```



Extraer memoria de un proceso

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 memdump -p 1234 -D ./proceso
```



Ver DLLs cargadas

```
python2.7 vol.py -f memoria.raw --profile=WinXPSP  
2x86 dlllist
```



Buscar strings en memoria

```
strings -a -td memoria.raw > strings.txt
```

CASO PRÁCTICO DE VOLATILITY 2



1. Dirección IP local de la víctima

```
vol.py -f estafabancaria.vmem imageinfo  
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 connections
```



2. Valor de la variable de entorno del SO

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 envvars | grep -i os
```



3. Contraseña del Administrador

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 hashdump
```

Normalmente utilizaríamos John the Ripper, pero como vamos con prisa y dos de los hashes NTLM nos suenan, usamos <https://crackstation.net/>.



4. Proceso responsable del exploit inicial

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 psxview
```



5. Extensión del archivo malicioso

El proceso sospechoso se hace pasar por Acrobat Reader, por lo que es muy probable que la extensión del exploit sea PDF.



6. IP externa sospechosa

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 connections  
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 pstree
```



7. URL PHP en memoria de svchost.exe

```
strings estafabancaria.vmem | grep "^http://" |  
    sort | uniq | grep "php"
```



8. Hash MD5 y fecha de envío a VirusTotal

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x
      86 memdump -p 1752 -D .
foremost 1752.dmp
cd output/pdf
md5sum * | grep 528afe08e437765cc
```

Consulta la fecha del primer envío en VirusTotal.



9. PID del proceso que cargó PDF.php

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 psxview
```



10. Función que oculta eval()

```
peepdf -i 00601560.pdf  
PPDF> object 11  
PPDF> object 1054
```

Vamos a <https://beautifier.io> y pega el JavaScript extraído.



10.b Instalando peepdf

```
pip2 install pycrypto  
git clone https://github.com/jesparza/peepdf.git
```



11. Shellcode para Acrobat v9

```
js-ascii js_dumped.js
```



12. Malware en winlogon.exe

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
      86 malfind -p 644 -D ./maldump/  
md5sum ./maldump/*
```

El archivo fue enviado a VirusTotal el 29/03/2010 a las 11:34:01.



13. Ejecutable malicioso (ZeuS)

```
vol.py -f estafabancaria.vmem --profile=WinXPSP2x  
86 filescan | grep -i ".exe"
```

Busca ejecutables sospechosos y búscalos en Google.



14. URL de descarga del archivo e.exe

```
strings estafabancaria.vmem | grep "http://" |  
    sort | uniq
```



15. CVE explotado por Acrobat v8

Podemos utilizar `jsunpack` para averiguar qué CVE se está explotando. Usamos el toolkit Cincan

(<https://cincan.io/>):

```
docker run -v $(pwd):/data cincan/jsunpack-n -v  
data/00601560.pdf -V -d /data/
```